23 · Ph2 交付：备份、决策会诊、MCP、事件管线

项	内容
文档编号	DT-PdM-23
日期	2026-10-22
版本	v0.4.2
测试	E2E 44/44 通过（连续两次）；单测 72 项

---

1. 备份（最高优先项，已完成）

项	结果
方式	TiDB 原生 `BACKUP DATABASE` → S3（MinIO）
目标	`s3://pdm-backup/full-20261022`（endpoint: milvus-minio.infra:9000）
规模	4,289,573,851 字节（4.29 GB），1 亿行全量
耗时	60.8 秒
校验	返回 backup-ts `469926584069128193`，与集群 TSO 一致

1.1 过程中解决的三个问题

#	问题	解法
1	`BACKUP ... TO 'local:///...'` 报 `File or directory not found on TiKV Node`	local:// 要求路径在 TiKV 容器内可见；改用 S3，跨集群可恢复
2	`bucket not found`	MinIO 无 pdm-backup 桶；用纯标准库实现 AWS SigV4（`tools/s3_mkbucket.py`）建桶，避免在容器内装 boto3
3	MinIO 凭据	从 `infra/platform-secrets` 读取（minioadmin/minioadmin123）

> 注意：备份是"一次性"的。定时备份尚未配置（需 CronJob + 保留策略），列为待办。

---

2. 决策会诊（L4，已完成）

2.1 能力

`POST /v1/consult`：输入 `{tenant_id, asset_id}`，自动完成 诊断 → 四角色独立出意见 → 独立评估 → 方案卡。

角色	授权数据	输出
shopfloor 车间经理	运行数据 + 现场	现象、紧迫度
reliability 可靠性	关键度 + 历史故障	故障模式、影响面
production 生产	排产负荷 + 窗口	可用窗口、冲突

角色	授权数据	输出
maintenance 维修	RUL + 备件 + 人力	方案、备件、工期
evaluator 评估者	四方意见（不生成）	一致性、冲突、证据缺口

2.2 修复的两个真 bug

#	现象	根因	修复
1	红色设备被判成 `MONITOR`	用 `max(set(...), key=count)` 聚合，set 迭代无序，2:2 时结果随机	改为加权投票 + 严重度兜底（`aggregate_verdict`），并加领域不变量：红色不得判为仅监测，强制升级 `ACT_NOW`（`verdict_escalated=true`）
2	备件未到时仍推荐提前窗口	窗口未与 ETA 对齐	`earliest = max(now+1d, eta+1d)`，且备件迟到时追加"紧急调拨"备选（溢价 35%）

2.3 实测输出（FAN-00004，红色）

```
verdict: ACT_NOW | escalated: true
options: A-■■■■ 2026-10-25 08:00 ¥43,800■■■■■■
         B-■■■■ 2026-10-23 00:14 ¥45,270■■■■■■■■ 35%■
net_benefit: ¥100,200 | consistency: 0.45 | risk: ■
conflicts: ■■■■■■shopfloor ■■ 4h■reliability ■■■■ 168h
```

---

3. MCP 服务（已完成）

`POST /mcp`（JSON-RPC 2.0），协议版本 `2024-11-05`，5 个工具：

工具	作用	副作用
`get_asset`	设备台账	无
`assess_health`	HI + 等级 + Top3 维度	无
`diagnose_asset`	完整诊断（5 维 + 假设 + 证据）	无
`consult_maintenance`	四角色会诊 → 方案卡	无（不建工单）
`list_hot_assets`	需优先关注设备	无

为什么自己实现而不装官方 SDK：服务器出口 128KB/s，SDK 依赖链长；MCP 的 HTTP 传输就是 JSON-RPC 2.0，三个方法即可，契约与官方一致，任意 MCP 客户端可连。

错误码遵循规范：`-32001` 缺租户、`-32601` 方法不存在、`-32602` 参数/工具错误。

---

4. 事件管线（已完成，含 CQRS 闭环）

4.1 架构

```
■■■■■/■■■
   ■ events.emit()  CloudEvents 1.0
   ▼
Redis Stream  pdm:events:<tenant>   (MAXLEN ~1e6, ■■ 7 ■)
   ■ XREADGROUP ■■■■
   ▼
```

```
■■■■Python / Go ■■■■■■■■ read-model-v1■
    ■
    ▼
■■■ read_asset_health / open_alerts
```

4.2 为什么用 Redis Streams 而不是 Pulsar（当前）

项	结论
Pulsar 租户	✅ 已创建 `pdm` + `site-a`/`site-b`/`group`（cluster-175）
HTTP 生产者接口	❌ broker 3.x 上 `/ws/v2/producer/...` 返回 404
`pulsar-client`	❌ 仅 manylinux wheel，服务镜像基于 musl，装不上
决策	传输层抽象为 `Transport`，当前实现 `RedisStreamTransport`；切 Pulsar 只需新增一个实现类

4.3 Go 事件管线（真实编译部署）

项	结果
语言/依赖	Go 1.22 + go-redis v9 + go-sql-driver v1.8
编译	`CGO_ENABLED=0`，静态二进制 5.7MB
镜像	distroless-static:nonroot，8.33MB
部署	`pdm-prod/go-eventpipe`，1 副本，Running
实测日志	`projected health FAN-00004 hi=0.0`

> 工具链突破：之前判断"无外网"不准确——实测 `mirrors.aliyun.com` 下载 Go 包达 1.48 MB/s。已安装 Go 1.22.5 到 `/usr/local/go`，`GOPROXY=https://goproxy.cn` 可用。

---

5. 本轮修复的工程问题

#	问题	修复
1	遗留 FastAPI 版 `app/main.py` 与 Litestar 版并存	删除，测试同步移除
2	Litestar 把 `payload: dict` 当 query 参数（400）	改为 `await request.json()` 直接读体，零框架歧义
3	`events.init` 未进启动钩子（字符串替换未命中）	显式补入 `_on_startup`
4	两个投影器写同一读模型会重复计数	幂等键统一为 `read-model-v1`（按事件去重，不按消费者）
5	L2 缓存读超时 0.2s，与事件写入争抢时偶发退化重算	超时放宽到 1.0s + 一次重试；E2E 从 43/44 稳定到 44/44
6	`/events/stats`、`/cache/stats` 是单 Pod 计数，负载均衡下读不准	文档标注；以 Redis/DB 为真值源

---

6. 当前系统全貌

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■■  Ingress /pdm-api → python-diagnosis (2 ■■)      ■
■      /healthz /version /v1/diagnose /v1/consult         ■
■      /mcp /warmup /cache/stats /events/stats            ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■■   pdm_signal■■■■■ + decision■■■■ + mcp        ■
■ ■■   L1 TTL30s + L2 Redis 120s + warmup■1201 ■/20s■     ■
■ ■■   Redis Stream → go-eventpipe■CQRS ■■■            ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■■   TiDB 8.5.8■PD3/TiKV3/TiDB2/TiFlash1■200Gi/■■■    ■
■      18 ■■ · 1 ■■■■■ · ■■ 4.29GB → MinIO        ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

---

7. Rust 边缘采集（已完成）

7.1 为什么零依赖

工业边缘要长期稳定运行，依赖越少越可靠；现场部署常无外网，`cargo` 拉 crates 是负担。因此只用 `std`：手写 HTTP/1.1（`TcpStream`）+ 自实现 xorshift 随机 + 纯 Rust DSP。

7.2 实测

```
[edge] target=10.167.2.175:32231 /pdm-api/v1/diagnose tenant=site-a asset=FAN-00004 fs=25600
[edge] #0 local_dsp rms=1.430 peak=2.399 kurt=1.58 crest=1.68 impulse=false
[edge] #0 uplink ok hi=100.0 level=green pending=0
[edge] #2 local_dsp rms=2.015 peak=6.101 kurt=1.60 crest=3.03 impulse=true
[edge] #2 uplink ok hi=99.77 level=green pending=0
[edge] #3 local_dsp rms=2.301 peak=7.271 kurt=1.59 crest=3.16 impulse=true
[edge] #3 uplink ok hi=95.21 level=green pending=0
```

三个能力被验证：

- 边缘本地 DSP：RMS/峰值/峭度/峰值因子在边缘算完再上行（原始波形不出厂）
- 冲击检出：注入点蚀后 crest 从 1.68 → 3.16，HI 从 100 → 95.21
- 断网降级：首次运行（无 Host 头）4 次全部 `DEGRADED` 且 `cached` 累加，恢复后 `pending=0`

7.3 工具链结论修正

判断	实测
"服务器无外网"	错误。`mirrors.aliyun.com` 实测 1.48 MB/s，`rsproxy.cn` 1.15 MB/s
Go	已装 1.22.5，`goproxy.cn` 可用
Rust	已装 1.99.1（rustup minimal），`rsproxy.cn` 可用
编译	`rustc -O` 单文件编译，无 cargo 依赖，4.5MB 静态二进制

> 之前"128KB/s"的判断只对某些 CDN 节点成立，不代表全部出口。结论：该服务器外网可用，限速是节点相关而非全局。

7.1 仍未完成

#	项	状态	说明
1	定时备份（CronJob + 保留策略）		当前仅一次性备份
2	Rust 边缘采集	已完成	见 §8（零依赖 std 实现，4.5MB 二进制，已跑通边缘→云端）

#	项	状态	说明
3	Java 资产/知识/集成		无 JDK；Nacos 已就绪，待 JDK 安装
4	方案卡落库 + 工单闭环		表结构已就绪（recommendation_card），服务未写入
5	Pulsar 后端		租户已建，客户端受阻（musl/glibc）
6	前端看板		无 UI，仅 API